# Simple DACS Forum - Security Analysis Report

Antoni Sienkiewicz i6367722

Julia Całuch i6364369

Group 25

**Introduction** 2
Project Objective 2
Scope 2
**Technologies Used** 3
**Intentional Vulnerabilities** 3
Weakness 1: SQL Injection 3
Weakness 2: Sensitive Data Exposure 4
Weakness 3: Broken Access Control 5
Weakness 4: Security Misconfiguration 6
Weakness 5: Broken Authentication 6
Weakness 6: Cross-Site Scripting 6
**Exploitation Phase** 7
1. SQL Injection Exploitation 7
2. Sensitive Data Exposure 7
3. Broken Access Control Exploitation 8
4. Security Misconfiguration Exploitation 9
6. Cross-Site Scripting (XSS) Exploitation 9
**Defense & Security Fixes** 10
1. SQL Injection Prevention 10
3. Access Control Enhancements 12
4. Security Configuration Improvements 14
5. Brute-Force Protection 16
6. Cross-Site Scripting (XSS) Mitigation 17
Implementation: 18
Implementation Results 19
**Threat Modeling** 19
Threat Actors 19
Attack Scenarios 19
Mitigation Strategies 19
**Future Security Improvements** 20
**Conclusion** 20
**Key Learnings** 20
**Challenges** 20
**References** 21

# Introduction

## Project Objective

This project aims to develop a simple forum website for DACS students, featuring user authentication (signup/login), post creation, and post deletion. The primary goal is to intentionally introduce security vulnerabilities, analyze real-world threats, exploit them for testing, and then implement security fixes. By adopting a defensive security approach, this project simulates real-world cybersecurity risks and provides hands-on experience in securing web applications.

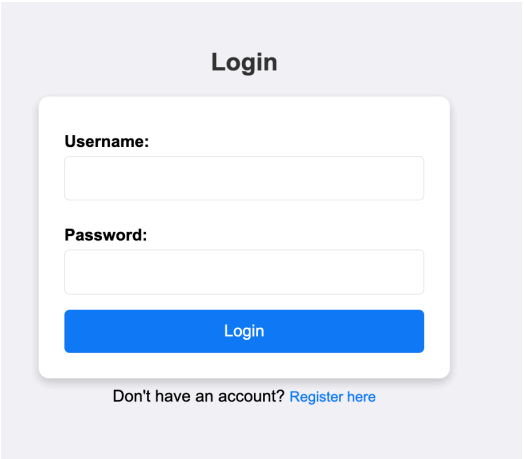
## Scope

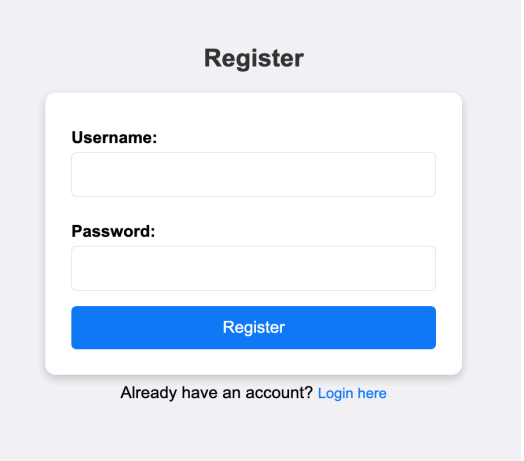
The forum is a basic web platform where students can create and manage posts. The focus is on identifying and mitigating common web security vulnerabilities, including weak authentication mechanisms, SQL injection risks, and improper access controls. Through this process, security flaws will be addressed by implementing appropriate countermeasures to enhance system protection.

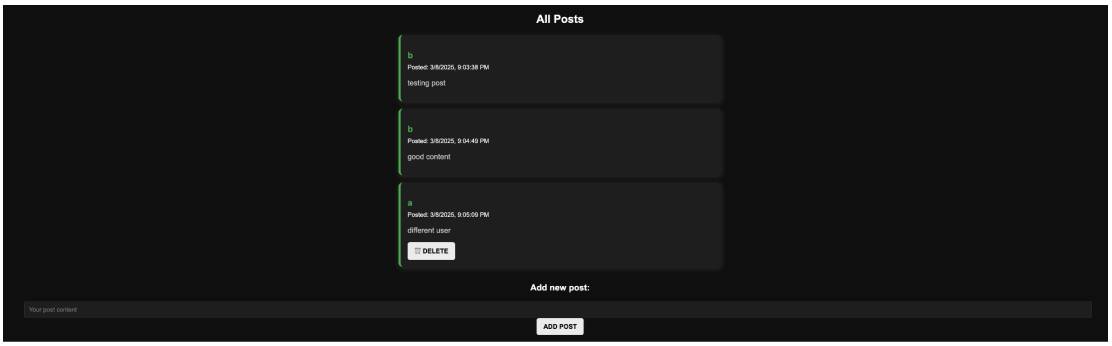
The project follows a three-phase approach:

1. Implementation of the forum with intentional vulnerabilities.
2. Exploitation of these vulnerabilities using common security tools and techniques.
3. Applying security fixes and validating their effectiveness.

Main functionalities:



# Technologies Used

The project is developed using the following technologies:

- **Frontend:** HTML, CSS (basic user interface for login, registration, and forum interactions).
- **Backend:** Java (Javalin framework for handling API endpoints and business logic).
- **Database:** MySQL (stores user credentials and forum posts).
- **Hosting Environment:** Raspberry Pi (provides a controlled lab setup to test security vulnerabilities)

# Intentional Vulnerabilities

To simulate real-world security flaws, the following 6 OWASP [1] vulnerabilities were deliberately introduced:

### Weakness 1: SQL Injection

The system is vulnerable to SQL injection because it directly inserts user input into SQL queries instead of using prepared statements [1]. This allows attackers to manipulate queries by injecting malicious code, such as ' OR '1'='1' --, which can bypass authentication and grant unauthorized access. Since there's no input validation, attackers can extract user credentials, modify data, or even delete records. Because the database processes user input as part of the query, it blindly executes malicious statements. Without query parameterization, injected code is treated as legitimate SQL, making it easy for attackers to expose sensitive information or alter database contents.

Current implementations of significant parts of code:

```
/**
* Validates user login (Completely Unsafe).
* @param username Username.
* @param password Password.
* @return true if credentials are correct, false otherwise.
*/
public boolean validateLogin(String username, String password) {
   if (username == null || password == null) {
       return false;
   }

   String query = "SELECT * FROM users WHERE username = '" + username + "' AND
password = '" + password + "'";

   try (Connection conn = DatabaseManager.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query)) {

       if (rs.next()) {
           return true;
```

```
    }

   } catch (SQLException e) {
       System.err.println("Error during login: " + e.getMessage());
   }
   return false;
}
```

### Weakness 2: Sensitive Data Exposure

During login, all user data, including passwords, are stored in plaintext, creating a great security risk. This practice makes the system highly vulnerable to brute-force attacks, as attackers can easily retrieve and test credentials. Additionally, internal threats become a significant problem, as any insider with database access can view and misuse sensitive user information. Without encryption or hashing, stolen credentials can be directly exploited without further decryption efforts.

Current implementations of significant parts of code:

```
public boolean addUser(User user) {
   String query = "INSERT INTO users (username, password) VALUES (?, ?)";

   try (Connection conn = DatabaseManager.getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {

       stmt.setString(1, user.username());
       stmt.setString(2, user.password());

       int rowsAffected = stmt.executeUpdate();
       return rowsAffected > 0;

   } catch (SQLException e) {
       System.err.println("Error while creation of a new user: " +
e.getMessage());
       return false;
   }
}
```

```
public boolean registerUser(String username, String password) {
   if (userDAO.getUserByUsername(username) != null) {
       return false;
   }

   User newUser = new User(username, password, 0);
   return userDAO.addUser(newUser);
}
```

Screenshot of user credentials stored in the database:

| user_id | username | password |
|---|---|---|
| 5 | b | b |
| 6 | a | a |

### Weakness 3: Broken Access Control

The system lacks proper access control, allowing anyone to retrieve all JSON files containing posts without needing to log in. This exposes sensitive data to unauthorized access and potential manipulation. Additionally, posts can be deleted simply by entering the correct post ID through terminal commands, without verifying whether the user actually owns the post. This means an attacker could delete other users' content, leading to privilege escalation. Without authentication and role-based permissions, the system is vulnerable to data tampering, content deletion, and censorship. To fix this, user authentication and strict access control checks must be enforced before allowing users to view or delete posts.

Current implementations of significant parts of code:

```
/**
* Returns all the posts on the forum.
*/
public Handler getAllPosts = ctx -> {
   List<Post> posts = postService.getAllPosts();
   ctx.json(posts);
};
```

```
/**
* Deletes post.
*/
public Handler deletePost = ctx -> {
   String sessionId = ctx.cookie("sessionId");
   User user = SessionManager.getUser(sessionId);

   if (user == null) {
       ctx.status(403).result("User not logged in.");
       return;
   }

   int postId = Integer.parseInt(ctx.pathParam("id"));
   boolean success = postService.deletePost(postId);

   if (success) {
       ctx.result("Post deleted successfully.");
   } else {
       ctx.status(403).result("Unauthorized to delete this post.");
   }
```

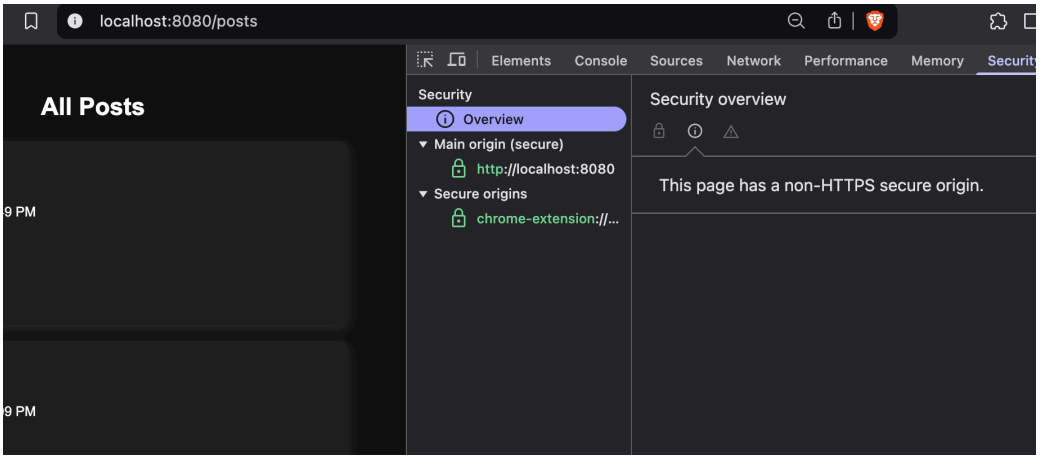
### Weakness 4: Security Misconfiguration

The system suffers from security misconfiguration due to the absence of HTTPS encryption and a secure lock when accessing the webpage. HTTPS is essential for encrypting data transmitted between users and the server, ensuring confidentiality and integrity. Without HTTPS, the application is vulnerable to man-in-the-middle (MITM) attacks, where attackers can intercept login credentials, session tokens, or other sensitive data.

Additionally, unencrypted connections expose users to session hijacking, where attackers can steal authentication cookies and impersonate users. The lack of a secure lock indicator in the browser also reduces user trust, making the application appear less credible. Without proper SSL/TLS configuration, data can be modified in transit, leading to potential security breaches.

Current implementations of significant parts of code:

```
// Creation of Javalin application
Javalin app = Javalin.create(config -> {
   config.jsonMapper(jsonMapper);
   config.staticFiles.add("/static", Location.CLASSPATH);
}).start(8080);
```




### Weakness 5: Broken Authentication

The authentication system has critical weaknesses that expose it to brute-force attacks and session hijacking risks. Currently, there is no login attempt limit or timeout for failed attempts, allowing attackers to repeatedly guess passwords without restriction. This significantly increases the risk of credential stuffing attacks, where stolen or commonly used passwords are tested against multiple accounts.

### Weakness 6: Cross-Site Scripting

The system lacks input sanitization and CSRF (Cross-Site Request Forgery) tokens, making it highly vulnerable to cross-site scripting (XSS) and CSRF attacks [4]. Without proper sanitization, user input can contain malicious scripts, allowing attackers to inject and execute JavaScript code within the application. This could lead to session hijacking, data theft, and phishing attacks [4].

Additionally, the absence of CSRF tokens means attackers can trick authenticated users into executing unintended actions, such as changing account settings or posting unauthorized content. This is particularly dangerous for users with higher privileges, as attackers could exploit their session to perform administrative actions.

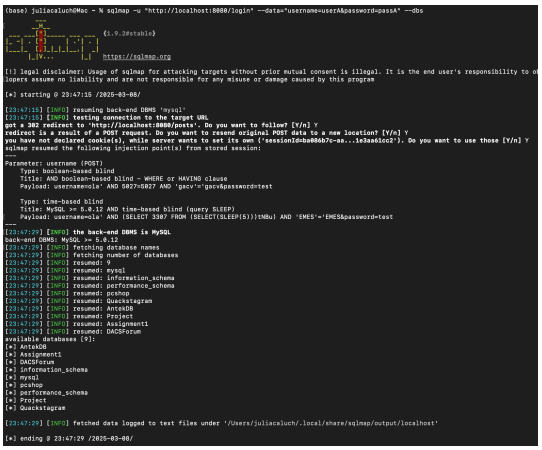
# Exploitation Phase

The exploitation phase focused on testing and demonstrating the vulnerabilities intentionally introduced in the system. We conducted attacks on our application, targeting SQL Injection, Sensitive Data Exposure, Broken Access Control, Security Misconfiguration, Broken Authentication, and Cross-Site Scripting (XSS). Below is a breakdown of our findings and exploitation attempts.

## 1. SQL Injection Exploitation

We tested the login functionality for SQL injection by injecting malicious SQL queries into the username and password fields. As user input was added directly into SQL queries without using prepared statements [1], we successfully bypassed authentication. Using SQLMap [7], we automated the attack and confirmed the vulnerability by detecting boolean-based blind SQL injection and time-based blind injection.

Additionally, we utilized UNION-based SQL injection through SQLMap [7] to retrieve confidential database entries, such as usernames and password hashes. The absence of proper input validation and query parameterization enabled us to alter queries and access information from the database structure.



## 2. Sensitive Data Exposure

In the previous section, we successfully retrieved all user credentials from the database using SQLMap [7], as passwords were stored in plaintext. The lack of password hashing or encryption meant that an attacker with database access could instantly obtain valid login credentials, significantly increasing the risk of account takeovers and credential leaks.

Additionally, the absence of rate limiting made the system vulnerable to brute-force attacks,allowing an attacker to repeatedly guess passwords without restriction. Combined with the ability to extract credentials via SQL injection, this flaw exposes user accounts, private data, and administrative access to potential attackers, posing a severe security risk.

```
Database: DACSForum
Table: users
[3 entries]
+----------+----------+
| username | password |
+----------+----------+
| a        | a        |
| b        | b        |
| UserA    | passA    |
+----------+----------+

[00:26:36] [INFO] table 'DAC
```

### 3. Broken Access Control Exploitation

We tested unauthorized post deletion and retrieval of posts. Using terminal commands [6], we performed the following actions:

1. Created a new user and logged in.
2. Created a post as the new user.
3. Retrieved all posts without authentication.
4. Deleted a post that belonged to another user.

The system allowed us to delete posts without verifying ownership, confirming that broken access control was a severe issue.

```
(base) juliacaluch@Mac ~ % curl -X POST http://localhost:8080/register -d
"username=UserA&password=passA"
Registration successful.%                                                (base)
juliacaluch@Mac ~ % curl -X POST http://localhost:8080/login -d
"username=UserA&password=passA" -c cookiesA.txt
Redirected%                                                              (base)
juliacaluch@Mac ~ % curl -X POST http://localhost:8080/api/posts --cookie cookiesA.txt -d
"content=This is UserA's post"
Post created successfully.%                                              (base)
juliacaluch@Mac ~ % curl -X GET http://localhost:8080/api/posts
[{"id":19,"username":"b","content":"testing post
","createdAt":1741464218000},{"id":20,"username":"b","content":"good content","(base)
juliacaluch@Mac ~ % (base) juliacaluch@Mac ~ (base) juliacaluch@Mac (base)
juliacaluch@Ma(base) j(base) j(base) j(base) j(base) juliacaluch(base) (b(ba(base) jul(base)
juliacal(base(base(base(base(base((base) juliacaluch@Mac ~ % curl -X DELETE
http://localhost:8080/api/posts/19 --cookie cookiesA.txt
Post deleted successfully.%
```

## 4. Security Misconfiguration Exploitation

The system was running without HTTPS, meaning all data transmissions were unencrypted. This made it vulnerable to man-in-the-middle (MITM) attacks, allowing attackers to intercept login credentials and session cookies.

Additionally, security headers were disabled, making the application more vulnerable to attacks such as cross-site scripting (XSS) and clickjacking.

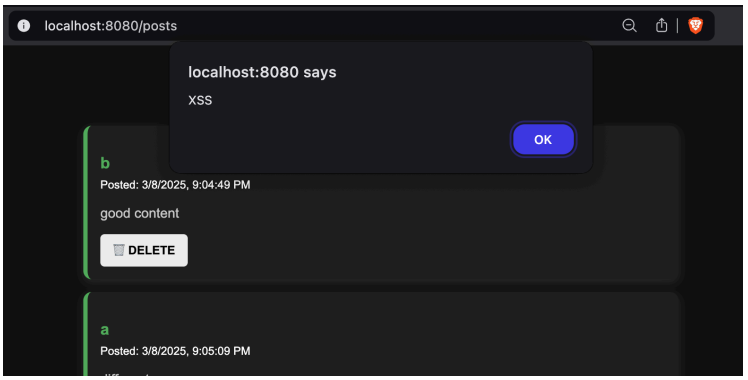
## 5. Broken Authentication Exploitation

The authentication system has critical weaknesses that expose it to brute-force attacks and session hijacking risks. Currently, there is no login attempt limit or timeout for failed attempts, allowing attackers to repeatedly guess passwords indefinitely. This significantly increases the risk of credential stuffing attacks, where stolen or commonly used passwords are tested against multiple accounts.

## 6. Cross-Site Scripting (XSS) Exploitation

To assess the vulnerability of our system to Cross-Site Scripting (XSS) attacks,we performed a set of tests following the methodologies outlined in Bright Security's XSS guide [5]. The purpose of these tests was to assess whether user input was adequately sanitized and if harmful scripts could run within the application.

We found that user input was not sanitized or encoded, enabling attackers to insert and run JavaScript code. This weakness was verified by multiple test cases, showing that the application was vulnerable to both stored and reflected XSS attacks. attacks.




<img src=x onerror=alert('XSS')>

<input type="text" value="XSS" onfocus="alert('XSS')">

# Defense & Security Fixes

Addressing the identified vulnerabilities required secure coding practices [2], proper authentication mechanisms, and security configuration improvements. The following fixes were implemented to mitigate the risks associated with SQL injection, weak password storage, access control flaws, misconfigured security settings, brute-force attacks, and cross-site scripting (XSS).

## 1. SQL Injection Prevention

To prevent SQL injection, we implemented prepared statements, ensuring user input is securely handled by binding values as parameters rather than concatenating queries. This prevents malicious input from altering SQL logic. Additionally, input validation restricts usernames to alphanumeric characters and underscores, blocking injection attempts. We also handle SQL integrity constraint violations, preventing errors from duplicate entries. These measures successfully mitigated SQL injection risks, ensuring secure database interactions.

Enhanced and secured version of code:

```
// Username pattern: Only letters, numbers, and underscores, between 3-20
characters
private static final Pattern USERNAME_PATTERN =
Pattern.compile("^[a-zA-Z0-9_]{3,20}$");

/**
* Adds a user to the database securely.
* @param user User object.
* @return true if registration is successful, false otherwise.
*/
public boolean addUser(User user) {
   // Validate username before inserting into the database
   if (!isValidInput(user.username())) {
       System.err.println("Invalid username format.");
       return false;
   }

   String query = "INSERT INTO users (username, password) VALUES (?, ?)";

   try (Connection conn = DatabaseManager.getConnection();
        PreparedStatement stmt = conn.prepareStatement(query)) {

       stmt.setString(1, user.username());
       stmt.setString(2, user.password());

       return stmt.executeUpdate() > 0;

   } catch (SQLIntegrityConstraintViolationException e) {
       System.err.println("Error: Username already exists.");
   } catch (SQLException e) {
       System.err.println("Error adding user: " + e.getMessage());
   }
   return false;
```

```
/**
* Validates input to prevent SQL Injection and invalid usernames.
* @param input User input.
* @return true if input is valid, false otherwise.
*/
private boolean isValidInput(String input) {
   return input != null && USERNAME_PATTERN.matcher(input).matches();
}
```

Results:

```
(base) juliacaluch@Mac ~ % sqlmap -u "https://localhost:8443/api/posts" \
    --cookie="sessionId=$3bf61ac8-f513-4af0-88e7-feb482e41930" \
    --batch --level=5 --risk=3 --dbms=mysql --random-agent
```

```
[16:29:28] [INFO] testing 'MySQL < 5.0.12 time-based blind - ORDER BY, GROUP BY clause (BENCHMARK)'
[16:29:28] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[16:29:29] [INFO] testing 'Generic UNION query (random number) - 1 to 10 columns'
[16:29:29] [INFO] testing 'MySQL UNION query (NULL) - 1 to 10 columns'
[16:29:30] [INFO] testing 'MySQL UNION query (random number) - 1 to 10 columns'
[16:29:31] [WARNING] parameter 'Host' does not seem to be injectable
[16:29:31] [CRITICAL] all tested parameters do not appear to be injectable. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=s
pace2comment')
[16:29:31] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 13614 times, 404 (Not Found) - 3717 times

[*] ending @ 16:29:31 /2025-03-09/
```

## 2. Secure Password Storage

To improve password security, the system was updated to securely store passwords with BCrypt hashing. In contrast to plaintext storage that leaves user credentials vulnerable to data breaches, BCrypt hashing guarantees that passwords are safely encrypted and protected against brute-force attacks.

### Implementation Details

To enhance password security, we replaced plaintext storage with BCrypt hashing, ensuring that passwords are securely transformed before being saved in the database. This means that even if the database is compromised, attackers cannot retrieve the actual passwords.

During login, the system uses BCrypt's verification function to securely compare the entered password with its hashed version, preventing unauthorized access. To strengthen security even more while maintaining efficiency, we set the cost factor to 12, making brute-force attacks significantly more difficult without compromising system performance.

Enhanced and secured version of code:

```
/**
* Registers a new user with secure password hashing.
*/
public boolean registerUser(String username, String password) {
   User existingUser = userDAO.getUserByUsername(username);
   System.out.println("Checking if user exists: " + username);
```

```java
    if (existingUser != null) {
        System.out.println("User already exists! Registration failed.");
        return false;
    }

    // Password hashing using BCrypt
    String hashedPassword = BCrypt.hashpw(password, BCrypt.gensalt(12)); // 12 =
cost factor
    User newUser = new User(username, hashedPassword, 0);
    boolean success = userDAO.addUser(newUser);
    System.out.println(success ? "User registered successfully!" : "User
registration failed.");
    return success;
}
```

Screenshot of user credentials stored in the database:

| user_id | username | password | created_at |
|---|---|---|---|
| 8 | pola | $2a$12$SVh3NKQ6WNBUAiB/OYBhYunPZuwz6uDPaCZwL4jgcYotWzs8DuDJ2 | 2025-03-09 14:32:42 |
| 9 | kuba | $2a$12$n59xYKHjl4rxtc9PdJHVbOWKXkK3.OBOUBC4R6u/lrFd/k6txyr1C | 2025-03-09 14:37:57 |
| 10 | antek | $2a$12$5wg0O6.D.JT4znbVUfAuc.D1w2wo6lCSGWAnDzEpp.Ez0mGltBq2O | 2025-03-09 14:38:07 |
| 11 | kamila | $2a$12$oOP3q3eDwRlkfjtM3CQpzeqbi7sAK2ppbHowRTfhttgdjt9jK6x0u | 2025-03-09 14:38:21 |

These measures significantly enhance the security of user credentials, protecting them from unauthorized access and reducing the risk of credential-based attacks.

## 3. Access Control Enhancements

To ensure that only authorized users could perform certain actions, improvements were made to user authentication and session management.

**Implementation:**

The system now verifies the logged-in user before allowing any actions by checking their session ID, which serves as a unique identifier for their login session. If no valid session ID is found, access to certain features is denied, preventing unauthorized actions.

Additionally, user role-based restrictions have been implemented to ensure that certain actions, such as deleting a post, can only be performed by the user who created it. Before allowing deletion, the system verifies whether the session ID matches the original post owner, preventing unauthorized modifications.

We also added the additional security level by adding CSRF tokens, but it is going to be discussed in the section about XSS.

This code prevents from getting JSON file with posts:

```
/**
* Gets all the posts on the forum.
*/
public Handler getAllPosts = ctx -> {
   String sessionId = ctx.cookie("sessionId");
   User user = SessionManager.getUser(sessionId);

   if (user == null) {
       ctx.status(403).json(Map.of("error", "User not logged in"));
       return;
   }

   List<Post> posts = postService.getAllPosts();
   ctx.json(posts);
};
```

This part of code is responsible for checking if the correct user is trying to delete the post:

PostService class:

```
public boolean deletePost(int postId, String username) {
   Post post = postDAO.getPostById(postId);
   if (post == null || !post.username().equals(username)) {
       return false; // Deny deletion if the user does not own the post
   }
   return postDAO.deletePost(postId);
}
```

ForumController class:

```
/**
* Deletes posts if it belongs to a legitimate user.
*/
public Handler deletePost = ctx -> {
   String sessionId = ctx.cookie("sessionId");
   User user = SessionManager.getUser(sessionId);

   if (user == null) {
       ctx.status(403).result("User not logged in.");
       return;
   }

   // Download and verification of CSRF token.
   String csrfTokenForm = ctx.formParam("csrfToken");
   String csrfTokenSession = ctx.sessionAttribute("csrfToken");

   if (csrfTokenForm == null || csrfTokenSession == null ||
!csrfTokenForm.equals(csrfTokenSession)) {
       ctx.status(403).result("CSRF protection triggered.");
       return;
   }
```

```
    ctx.sessionAttribute("csrfToken", null); // Deletion of token after usage.

    int postId = Integer.parseInt(ctx.pathParam("id"));
    boolean success = postService.deletePost(postId, user.username());

    if (success) { //Deny to perform deletion if the user is not the one that
created the post.
        ctx.result("Post deleted successfully.");
    } else {
        ctx.status(403).result("Unauthorized to delete this post.");
    }
};
```

Results of trying to get the data again:

```
(base) juliacaluch@Mac ~ % curl -k -X POST https://localhost:8443/register -d
"username=UserA&password=passA"
Registration successful.%
(base) juliacaluch@Mac ~ % curl -k -X POST https://localhost:8443/login -d
"username=UserA&password=passA" -c cookiesA.txt
Redirected%
(base) juliacaluch@Mac ~ % curl -k -X GET https://localhost:8443/api/posts
{"error":"User not logged in"}%
(base) juliacaluch@Mac ~ % curl -k -X DELETE https://localhost:8443/api/posts/37
--cookie cookiesA.txt
CSRF protection triggered.%
```

These measures help protect user data and ensure that only authorized individuals can make changes.

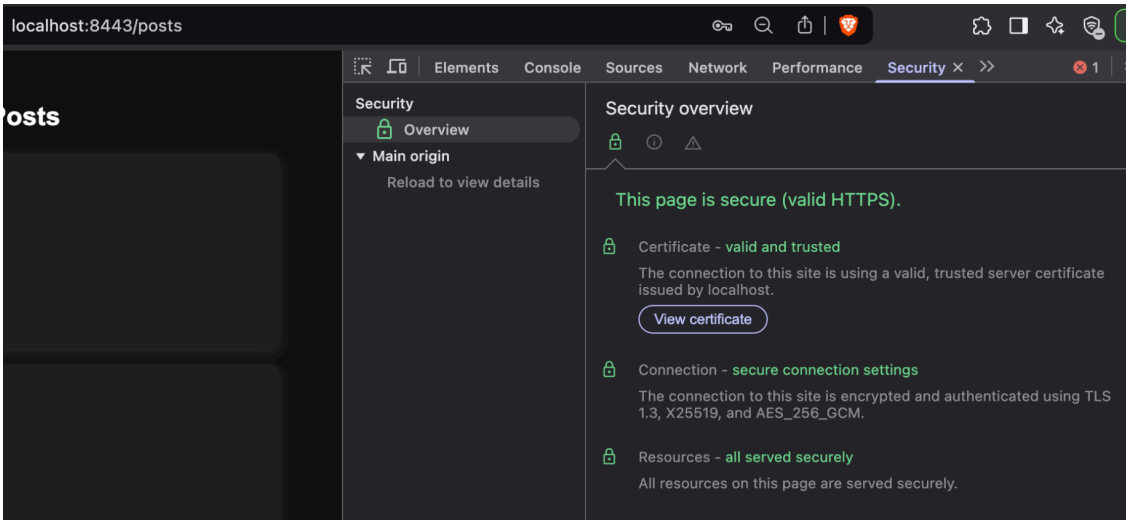
## 4. Security Configuration Improvements

The system originally did not have HTTPS encryption, exposing login details and session tokens to MITM attacks. To address this, we created an SSL/TLS certificate and set up the server to require HTTPS, guaranteeing that all interactions between users and the server are secure.

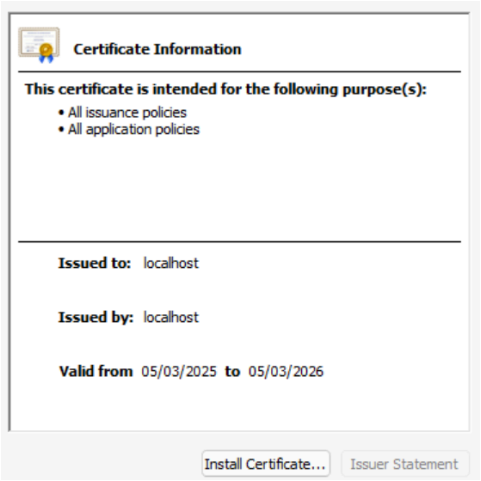
In the absence of HTTPS, user sessions were vulnerable to being hijacked, enabling attackers to impersonate users. We addressed this by acquiring an SSL certificate from a reputable Certificate Authority (CA) and redirecting all HTTP traffic to HTTPS, thereby preventing session hijacking.

Moreover, the absence of the secure lock icon in the browser reduced user trust. By adopting HTTPS with a legitimate certificate, the system now guarantees data integrity, avoids tampering, and improves security.

Result:




Certificate:




Main class:

```
Javalin app = Javalin.create(config -> {
   config.staticFiles.add("/static", Location.CLASSPATH); // CSS & JS load
   config.jetty.server(() -> {
       Server server = new Server();

       //HTTP Connector (Redirection HTTP to HTTPS)
       ServerConnector http = new ServerConnector(server);
       http.setPort(8080);

       //SSL Context
       SslContextFactory.Server sslContextFactory = new
SslContextFactory.Server();

sslContextFactory.setKeyStorePath(Paths.get("keystore.jks").toAbsolutePath().toS
tring());
       sslContextFactory.setKeyStorePassword("Frombork132");
       sslContextFactory.setKeyManagerPassword("Frombork132");
```

```
        sslContextFactory.setCertAlias("selfsigned");

        //Ensure Proper TLS and Cipher Settings
        sslContextFactory.setIncludeProtocols("TLSv1.2", "TLSv1.3");

sslContextFactory.setExcludeCipherSuites("TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256"
); // Exclude weak cipher
        sslContextFactory.setEndpointIdentificationAlgorithm(null);

        //SecureRequestCustomizer Fix
        HttpConfiguration httpsConfig = new HttpConfiguration();
        httpsConfig.addCustomizer(new SecureRequestCustomizer());
        httpsConfig.setSecureScheme("https");
        httpsConfig.setSecurePort(8443);

        ServerConnector https = new ServerConnector(
                server,
                new SslConnectionFactory(sslContextFactory, "http/1.1"),
                new HttpConnectionFactory(httpsConfig)
        );
        https.setPort(8443);

        server.setConnectors(new Connector[]{http, https});
        return server;
    });
}).start();
```

## 5. Brute-Force Protection

To mitigate brute-force attacks, a login attempt limitation mechanism was implemented. After five failed attempts, the account is temporarily locked to prevent automated attacks.

**Implementation:**

- Limited failed login attempts to five retries, after which the account is temporarily locked.
- Implemented an exponential backoff mechanism for reattempting logins.

AuthService Class:

```
/**
* Logs in a user and prevents brute force attempts.
*/
public String loginUser(String username, String password) {
    // Check if user is blocked due to too many failed attempts
    if (isUserBlocked(username)) {
        System.out.println("User " + username + " is temporarily blocked from
logging in.");
        return null;
    }

    User user = userDAO.getUserByUsername(username);
    if (user != null && BCrypt.checkpw(password, user.password())) { //Secure
```

```
password check
        // Reset failed attempts on successful login
        failedAttempts.remove(username);
        blockedUsers.remove(username);

        return SessionManager.createSession(user);
    } else {
        // Increase failed attempts
        failedAttempts.put(username, failedAttempts.getOrDefault(username, 0) +
1);

        // Block user if limit is reached
        if (failedAttempts.get(username) >= MAX_ATTEMPTS) {
            blockedUsers.put(username, System.currentTimeMillis() +
BLOCK_TIME_MS);
            System.out.println("User " + username + " is now blocked for 15
minutes.");
        }
        return null;
    }
}
```

Result:

```
[main] INFO io.javalin.Javalin - Javalin started in 378ms \o/
HTTPS Server running at https://localhost:8443
pola failed to login!
pola failed to login!
pola failed to login!
pola failed to login!
pola failed to login!
User pola is now blocked for 15 minutes.
User pola is temporarily blocked from logging in.
```

## 6. Cross-Site Scripting (XSS) Mitigation

To protect against XSS attacks, we sanitize user input before storing it, stopping harmful scripts from running on web pages. This is achieved by escaping HTML characters, making sure that any possibly dangerous code is handled as plain text instead of executing in the browser. This method aids in avoiding stored and reflected XSS vulnerabilities.

To further improve security, we also implemented CSRF tokens, which ensure that actions like form submissions and requests come from authenticated users rather than malicious attackers.

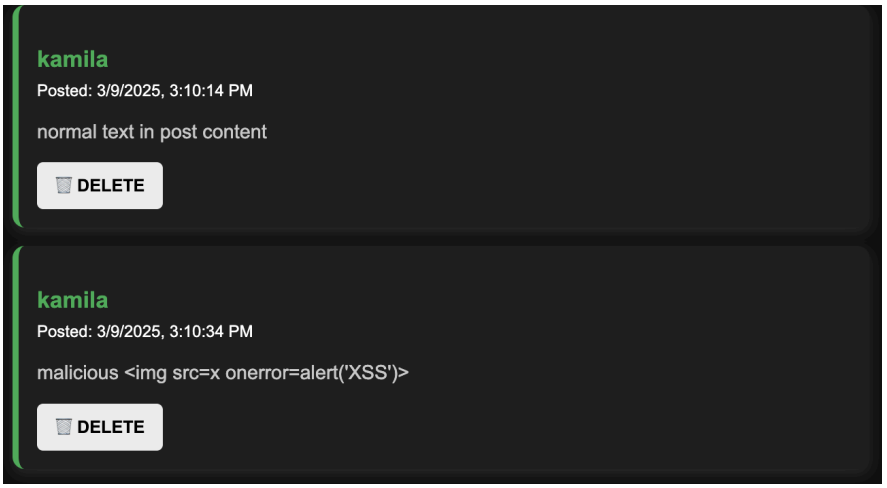
## Implementation:

When a user submits content, HTML characters are escaped before being stored in the database. This prevents scripts from executing when the content is later displayed. Although this eliminates harmful code, it does not fully removes it. Additionally, CSRF tokens are generated and verified for each user session to prevent unauthorized requests.

These measures significantly reduce the risk of XSS attacks and unauthorized actions, making the system more secure for users.

PostService class:

```
public boolean createPost(String username, String content) {
   if (content == null || content.trim().isEmpty()) {
       return false;
   }
   String sanitizedContent = StringEscapeUtils.escapeHtml4(content);//sanitizes
text in post
   System.out.println("User: " + username + "added post with content: " +
sanitizedContent);
   Post newPost = new Post(0, username, sanitizedContent, new
Timestamp(System.currentTimeMillis()));
   return postDAO.insertPost(newPost);
}
```




Debugging statements:

```
User: kamilaadded post with content: normal text in post content
User: kamilaadded post with content: malicious &lt;img src=x onerror=alert('XSS')&gt;
```

Enhanced and secured version of code creating CSRF tokens:

```
public Handler getCSRFToken = ctx -> {
    String csrfToken = UUID.randomUUID().toString();  // Generate a
unique token
    ctx.sessionAttribute("csrfToken", csrfToken);  // Store token in
```

```
session
    ctx.json(Map.of("csrfToken", csrfToken));  // Send token as JSON
response
};
```

```
String csrfTokenForm = ctx.formParam("csrfToken");  // Get token from
request
String csrfTokenSession = ctx.sessionAttribute("csrfToken");  // Get
token from session

if (csrfTokenForm == null || csrfTokenSession == null ||
!csrfTokenForm.equals(csrfTokenSession)) {
    ctx.status(403).result("CSRF protection triggered. Invalid token.");
    return;
}
ctx.sessionAttribute("csrfToken", null);  // Invalidate the token after
use
```

### Implementation Results

After implementing these security improvements, re-exploiting the vulnerabilities that were identified earlier was unsuccessful. The efficacy of the security measures put in place was confirmed via penetration testing and code evaluations, guaranteeing the strength of the web application against typical security vulnerabilities

## Threat Modeling

### Threat Actors

The primary threat actors include malicious hackers, who may attempt to steal user data or disrupt forum operations through SQL injection, brute-force attacks, or session hijacking. Additionally, insider threats pose a risk, as users with database access could exploit weak security mechanisms for unauthorized actions.

### Attack Scenarios

Potential attacks include SQL injection to gain unauthorized access to user credentials, cross-site scripting (XSS) to steal session cookies, and broken access control exploits that allow unauthorized post deletion or data exposure. Additionally, the lack of HTTPS makes the system vulnerable to man-in-the-middle (MITM) attacks.

### Mitigation Strategies

To mitigate these threats, we implemented prepared statements to prevent SQL injection, password hashing (BCrypt) to secure credentials, and session management improvements were made to restrict unauthorized access. HTTPS enforcement and input sanitization were also applied to defend against MITM and XSS attacks.

# Future Security Improvements

To address security vulnerabilities, a number of essential measures should be put into action. Multi-Factor Authentication (MFA) provides an additional security layer, minimizing the chances of credential-focused attacks. Safe management of cookies, incorporating HttpOnly and Secure attributes, stops session hijacking and cross-site scripting (XSS) by limiting client-side access to session tokens. Requiring HTTPS-only transmission of cookies enhances session security even more.

Automated security testing aids in the early detection of vulnerabilities by incorporating security scanners and penetration testing frameworks into the development process, revealing threats such as SQL injection, improper access controls, and insecure API settings prior to deployment. Furthermore, improving password security by blacklisting weak passwords makes credentials less vulnerable to brute-force attacks.

Implementing these strategies allows web applications to effectively address typical threats, leading to enhanced data security, minimized unauthorized access, and improved system integrity.

# Conclusion

This project provided valuable insights into web security vulnerabilities and the importance of secure development practices. The main concerns included unauthorized backend access, insecure authentication mechanisms, and weak session management. Through implementation and testing, we identified security flaws and applied mitigation strategies to strengthen the system.

## Key Learnings

The project demonstrated how easily misconfigured security settings and weak authentication can be exploited to gain unauthorized access. Insecure backend implementations, such as unvalidated user inputs and weak password policies, significantly increase security risks. Understanding the frontend-backend interaction was crucial in identifying injection-based threats and data exposure issues.

## Challenges

We encountered multiple security vulnerabilities, with SQL injection being the most critical, allowing attackers to manipulate database queries. Broken access control and poor session management further compromised security, enabling unauthorized actions. Additionally, configuring HTTPS encryption and addressing user validation issues posed technical challenges, requiring rigorous testing and secure coding practices [2].

# References


1. OWASP. (n.d.). *OWASP top ten security risks*. OWASP. Retrieved March 6, 2025, from https://owasp.org/www-project-top-ten/
2. Oracle. (n.d.). *Java secure coding guidelines*. Oracle. Retrieved March 6, 2025, from https://www.oracle.com/java/technologies/javase/seccodeguide.html
3. Oracle. (n.d.). *MySQL prepared statements*. Oracle. Retrieved March 7, 2025, from https://dev.mysql.com/doc/refman/8.0/en/sql-prepared-statements.html
4. PortSwigger. (n.d.). *Cross-site scripting (XSS) vulnerabilities*. Retrieved March 7, 2025, from https://portswigger.net/web-security/cross-site-scripting
5. Bright Security. (n.d.). *Cross-site scripting (XSS): What it is & how to prevent it*. Retrieved March 8, 2025, from https://brightsec.com/blog/cross-site-scripting-xss/
6. Boolean World. (n.d.). *cURL command tutorial with examples*. Retrieved March 8, 2025, from https://www.booleanworld.com/curl-command-tutorial-examples/
7. **Vaadata.** (n.d.). *SQLMap: The tool for detecting and exploiting SQL injections*. Vaadata. Retrieved March 9, 2025, from https://www.vaadata.com/blog/sqlmap-the-tool-for-detecting-and-exploiting-sql-injections/